

that this is not a good idea since it allows unauthorised access to hardware. But that's not true. If you implement it in the proper way, the device can never be accessed wrongly by a program.

In Linux, you can find many drivers and they are identified by their unique major number. It is also interesting to note that since a particular DD may be used to control different physical and virtual devices (the HDD and partitions, for example), the individual device will be assigned a minor number (ranging from 0 to 255). You can comprehend this better if you look at the box titled 'Device driver nomenclature'.

Note the exceptions mentioned in the box. These exceptions are seen for DD corresponding to terminals and serial interfaces (which are assigned major numbers 4 and 5). Here, the devices with the number 4 are essentially virtual consoles, simple serial interfaces and pseudo-terminals. You may note that the virtual consoles are assigned the numbers ranging from 0 (which obviously corresponds to `tty0`) to 63 and `/dev/tty0` or `/dev/console` corresponds to the current virtual console.

For a serial interface there are two logical devices—`ttySn` (dial-in device) and `cuan` (call-out device). When `ttySn` is opened, the kernel restricts access to it for other programs till the DTR line is active. And when it comes to the accessing of `cuan`, the corresponding process will be provided with immediate access to the serial interface (provided it is not in use). This will keep on blocking any process that tries to use `ttySn` (assigned with minor numbers 64 to 127). You can also see that the system assigns the minor numbers from 128 to 255 for pseudo-terminals. The master terminal (`ptyn`) is assigned `128+n` and the corresponding slave (`tyyn`) is given `192+n`.

The major number 5 is assigned for the current terminal and call-out devices. `/dev/tty` is given the minor number 0. And the corresponding `cuan` devices are assigned with minor numbers `64+n`.

Here is another list that could be handy when you write DD to access some input devices:

- 11 char Raw keyboard device (Linux/SPARC only)
 - 0 = `/dev/kbd` (raw keyboard device)
- 11 char Serial Mux device (Linux/PA-RISC only)
 - 0 = `/dev/ttyB0` (first mux port)
 - 1 = `/dev/ttyB1` (second mux port)
- 11 block SCSI CD-ROM devices
 - 0 = `/dev/scd0` (first SCSI CD-ROM)
 - 1 = `/dev/scd1` (second SCSI CD-ROM)

Akin to the filesystem (you can guess why it is so!), the DD is required to be 'made known' to the kernel. This is made possible with the help of the driver modules that are initialised while booting the system. When you code, the following list of functions will be helpful while performing this:

```
int register_chrdev(unsigned int major, const char * name, struct
file_operations * fops);

int register_kbddev(unsigned int major, const char * name, struct
file_operations * fops);
```

Since we are dealing with device drivers, I think some of you might be interested in the special initialisers that we use

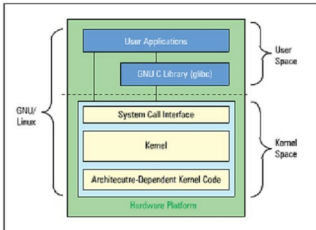


Figure 1: Tier architecture

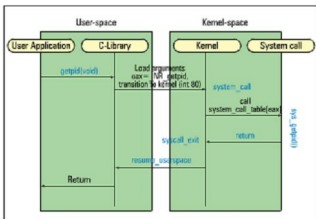


Figure 2: User space and kernel space

for USB Mass Storage devices:

```
#include <linux/sched.h>
#include <linux/errno.h>
#include "usb.h"
#include "initializers.h"
#include "debug.h"
#include "transport.h"

/* This places the Shuttle/SCM USB->SCSI bridge devices in null-target
 * mode */
int usb_stor_enusci_init(struct us_data *us)
{
    int result;

    US_DEBUGP("Attempting to init eUSCSI bridge.\n");
    us->iohub[0] = 0x1;
    result = usb_stor_control_msg(us, us->send_ctl_pipe,
                                0x0C, USB_RECIPIENT_INTERFACE | USB_TYPE_VENDOR,
                                0x01, 0x0, us->iohub, 0x1, 0x0);
    US_DEBUGP("result is %d\n", result);

    return 0;
}
```